Principios de Desarrollo de software

Pruebas de CRUD (Unitarias, Integración y Sistema)

Objetivo del Laboratorio:

El objetivo del laboratorio es que los estudiantes desarrollen y ejecuten pruebas de software de tres tipos (unitarias, integración y sistema) sobre un CRUD previamente implementado en el proyecto Zoo de Mascotas Fantásticas usando Spring Boot y MySQL.

Parte 1: Pruebas Unitarias

Introducción:

Las pruebas unitarias son esenciales para verificar el correcto funcionamiento de partes individuales del código.

Pasos a seguir:

1. **Implementación de Pruebas Unitarias:**
   - Crea pruebas unitarias para el servicio `CreatureService`, usando JUnit y Mockito. Debes cubrir las siguientes funciones:
     - **Crear criatura** (`createCreature`).
     - **Obtener criatura por ID** (`getCreatureById`).
     - **Actualizar criatura** (`updateCreature`).
     - **Eliminar criatura** (`deleteCreature`).
   - Crear las mismas pruebas para zonas.

Ejemplo de prueba unitaria para `createCreature`:

```
@Test
void testCreateCreature_ShouldReturnSavedCreature() {
    Creature creature = new Creature();
    creature.setName("Fénix");
    when(creatureRepository.save(any(Creature.class))).thenReturn(creature);

    Creature savedCreature = creatureService.createCreature(creature);

    assertNotNull(savedCreature);
    assertEquals("Fénix", savedCreature.getName());
}
```

2. **Pruebas adicionales:**
   - Implementa pruebas que verifiquen que **no se puede eliminar una criatura en estado crítico**.
   - Cubre también casos donde los parámetros de entrada son inválidos, usando las validaciones del modelo.

Parte 2: Pruebas de Integración

Introducción:

Las pruebas de integración validan cómo interactúan varios componentes entre sí. Usaremos una base de datos H2 para probar la persistencia de las criaturas.

Investigar que es una base de datos H2 y porque se utilizan para las pruebas de integración.

Pasos a seguir:

1. **Configuración de la base de datos H2:**
   - Añade la dependencia de H2 en `pom.xml`:

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

   - Configura el entorno de pruebas en `application.properties`:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
```

2. **Escribir las pruebas de integración:**
   - Implementa pruebas que verifiquen la interacción entre el servicio `CreatureService` y la base de datos. Asegúrate de cubrir al menos:
     - Crear criatura y verificar que se haya guardado en la base de datos.
     - Actualizar criatura y verificar los cambios en la base de datos.
     - Eliminar criatura y confirmar que ya no existe en la base de datos.

Ejemplo de prueba de integración para `createCreature`:

```
@SpringBootTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
public class CreatureServiceIntegrationTest {

    @Autowired
    private CreatureService creatureService;

    @Autowired
    private CreatureRepository creatureRepository;

    @Test
    void testCreateCreature_ShouldPersistInDatabase() {
        Creature creature = new Creature();
        creature.setName("Unicornio");
        creature.setDangerLevel(5);

        creatureService.createCreature(creature);
        Optional<Creature> foundCreature =  creatureRepository
                                            .findById(creature.getId());

        assertTrue(foundCreature.isPresent());
        assertEquals("Unicornio", foundCreature.get().getName());
    }
}
```

Parte 3: Pruebas de Sistema con Postman

Introducción:

Aunque ya se realizaron pruebas con Postman en el laboratorio anterior, esta sección está diseñada para que automatizen esas pruebas utilizando la funcionalidad de colecciones y tests en Postman. Esto les permitirá crear un conjunto repetible de pruebas que pueden ejecutar automáticamente para validar todo el CRUD.

Pasos a seguir:

1. Crear Colección de Pruebas en Postman:
   - Deberán crear una colección de Postman que agrupe las siguientes solicitudes:
     - POST: Crear una criatura.
     - GET: Obtener la lista de criaturas.
     - GET: Obtener una criatura por ID.
     - PUT: Actualizar una criatura.
     - DELETE: Eliminar una criatura (verificando que no esté en estado crítico).
2. Agregar Tests en Postman:
   - Por cada solicitud, los estudiantes agregarán scripts de tests en Postman para verificar que las respuestas sean correctas.

Ejemplo de un test de creación (POST):

```
pm.test("Creature created successfully", function () {
    pm.response.to.have.status(201);
    const jsonData = pm.response.json();
    pm.expect(jsonData.name).to.eql("Dragón");
});
```

Ejemplo de un test para obtener criatura por ID (GET):

```
pm.test("Creature retrieved successfully", function () {
    pm.response.to.have.status(200);
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("id");
    pm.expect(jsonData.name).to.eql("Dragón");
});
```

3. Encadenar solicitudes con variables:
   - Deben hacer que las pruebas sean dinámicas, usando variables para encadenar las solicitudes. Por ejemplo, guardar el ID de la criatura creada en una variable para luego usarlo en las solicitudes GET, PUT y DELETE.

Ejemplo de cómo almacenar el ID de una criatura creada:

```
const jsonData = pm.response.json();
pm.environment.set("creatureId", jsonData.id);
```

4. Ejecución de la Colección:
    - Deberán ejecutar toda la colección de pruebas en Postman, ya sea de manera manual o usando la funcionalidad de Runner para automatizar la ejecución de todas las solicitudes.
    - Postman debe mostrar un resumen de los resultados, indicando cuántas pruebas pasaron y cuántas fallaron.

5. Informe de Resultados:
    - Deberan guardar un reporte del resultado de la ejecución de la colección (Postman permite exportar reportes en formato JSON o HTML) y analizar cualquier error encontrado.

Material de Apoyo:

- **Documentación sobre Tests en Postman**

Parte 4: Pruebas de Rendimiento con JMeter

Introducción:

Para complementar las pruebas funcionales, ejecuta pruebas de rendimiento usando **Apache JMeter**. Estas pruebas evaluarán la capacidad del sistema para manejar múltiples solicitudes simultáneas (por ejemplo, cuando varios usuarios acceden al CRUD a la vez).

Pasos a seguir:

1. **Instalar JMeter:**
    - Descarga e instala Apache JMeter.
    - Familiarízate con la interfaz y los elementos clave: **Thread Groups**, **Samplers**, **Listeners**.
2. **Configurar un test de carga:**
    - Simula múltiples usuarios enviando solicitudes al endpoint de la API REST del CRUD (por ejemplo, `POST /api/creatures`).
    - Configura un **Thread Group** con 50 usuarios simulados que envían solicitudes de creación de criaturas.

3. **Análisis de resultados:**
    - Usa los **Listeners** para analizar el tiempo de respuesta, número de errores y el rendimiento bajo carga.
    - Compara los resultados obtenidos bajo diferentes configuraciones (10, 50 y 100 usuarios simultáneos).

Material de Apoyo:

- Guía de JMeter

Tips para el Informe:

1. **Resultados claros y organizados:**
    - Presenta los resultados de cada tipo de prueba (unitaria, integración, sistema) en secciones separadas. Utiliza capturas de pantalla y código relevante.
2. **Enfoque en la solución de problemas:**
    - Si encontraste errores durante las pruebas, documenta cómo los identificaste y qué soluciones aplicaste.
3. **Explicación de los casos de prueba:**
    - Describe brevemente los casos de prueba más importantes, qué validaron y por qué fueron necesarios.
4. **Análisis de desempeño:**
    - Si realizaste pruebas de rendimiento (con JMeter), incluye gráficos o tablas con el tiempo de respuesta y comenta sobre el comportamiento bajo diferentes cargas.
5. **Reflexión final:**
    - Resume los aprendizajes clave: ¿Qué desafíos enfrentaste al crear y ejecutar las pruebas? ¿Qué parte del proceso mejoró tu comprensión de los sistemas CRUD?